

Mini Game Hub

CS-108 Project

Yogi & Praneeth

April 2026

Abstract

This report is about the development of the Mini Game Hub, a secure, multi-user platform that combines Bash scripting for user authentication and Python (Pygame) for gameplay. Two authenticated players can select a game from the menu, play through a graphical interface, and save their results to a persistent leaderboard.

Contents

1	Introduction	2
2	Libraries and Tools	2
3	System Architecture	2
3.1	Directory Structure	2
4	Features	2
5	Core Implementation Logic	3
5.1	Authentication System (Bash)	3
5.2	Generic Game Class (Inheritance)	3
5.3	Game-Specific Logic	3
6	Leaderboard and Analytics	4
7	Hurdles and Solutions	4
8	The "10-Hour" Plan	5

1 Introduction

The Mini Game Hub is a pygame,bash platform where two players can log in and compete in various board games. The system's architecture focuses on two main areas: security (using SHA-256 hashing for passwords) and efficient game logic using NumPy matrix operations.

2 Libraries and Tools

- **Bash / Shell:** Used for the authentication of the system and formatting the terminal-based leaderboard.
- **Pygame-CE:** Used to build the GUI and handle real-time user input.
- **NumPy:** Used for board representations and win-condition slicing to keep the game logic fast and efficient.
- **Matplotlib:** Used to generate post-game analytics, including bar and pie charts.

3 System Architecture

3.1 Directory Structure

```
cs108-project/  
├── main.sh ... (Entry point & Auth)  
├── game.py ... (Main Menu & Selection)  
├── leaderboard.sh ... (Bash Statistics)  
├── matplot.py ... (Analytics)  
├── users.tsv ... (Hashed Credentials)  
├── history.csv ... (Game Records)  
├── games/  
│   ├── base_game.py ... (Parent Class)  
│   ├── tictactoe.py  
│   ├── othello.py  
│   └── connect4.py  
├── report.tex  
└── Makefile
```

4 Features

- **User Authentication:** A secure login system using SHA-256 hashing to protect the user passwords.
- **Multi-Game Support:** Includes full versions of Tic-Tac-Toe, Othello, and Connect 4.
- **Turn-Based Game play:** A reliable system for managing player turns and game flow.
- **Win Detection:** Optimized algorithms that instantly detect winning conditions.
- **Leaderboard System:** Tracks player wins and losses over time.
- **Data Persistence:** All match records are saved in CSV format.
- **Data Visualization:** Automatically generates and opens the performance graphs using Matplotlib.
- **Menu Interface:** A user-friendly GUI for easy navigation and game selection.

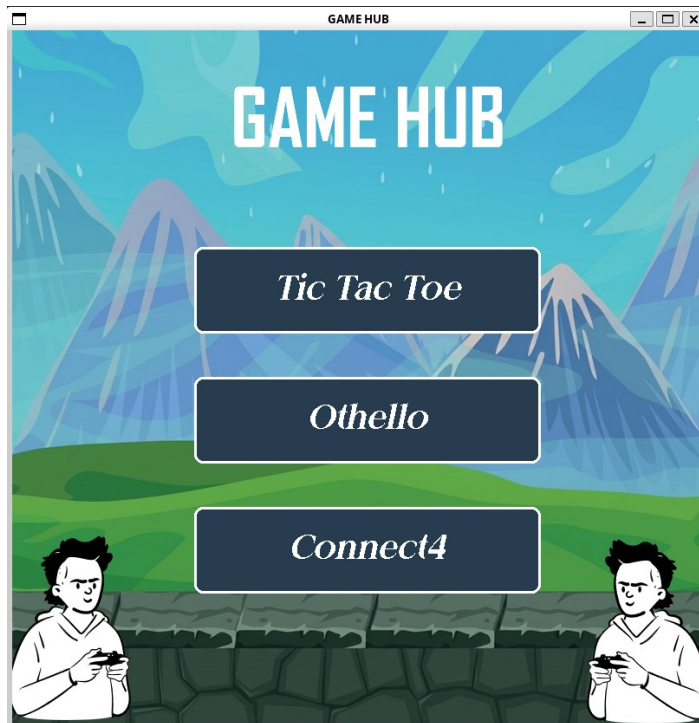


Figure 1: The Mini Game Hub Main Menu Interface

5 Core Implementation Logic

5.1 Authentication System (Bash)

The `main.sh` script acts as the entry point. It uses the `sha256sum` tool to hash passwords before storing them in `users.tsv`. This ensures that even if the data file is accessed, the actual passwords remain secure.

5.2 Generic Game Class (Inheritance)

We created a `Base` class to handle shared properties like player names, turn switching. Specific games like `Tictactoe` and others too inherit from this class.

5.3 Game-Specific Logic

- **Tic-Tac-Toe:** Played on a 10x10 grid where players must connect 5 marks. We used NumPy slicing for quick win-checking.
- **Othello:** Uses a "checkline" algorithm to detect valid moves and flip opponent pieces across rows, columns, and diagonals.
- **Connect 4:** Implements "gravity" logic to drop pieces to the lowest available row.

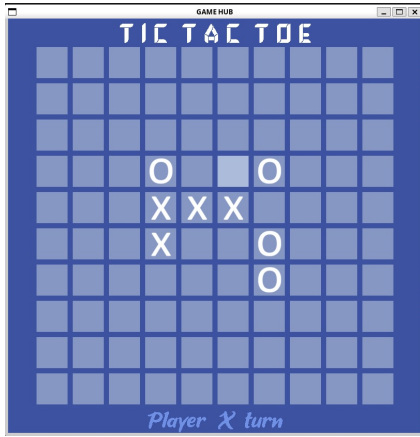


Figure 2: Tic-Tac-Toe

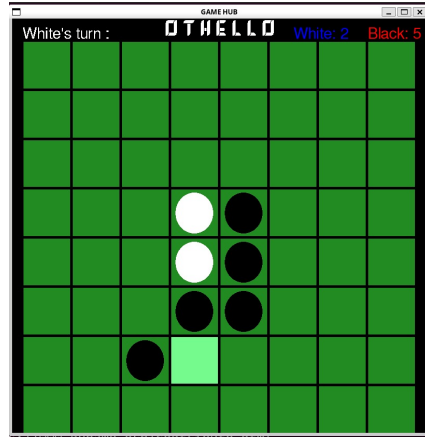


Figure 3: Othello

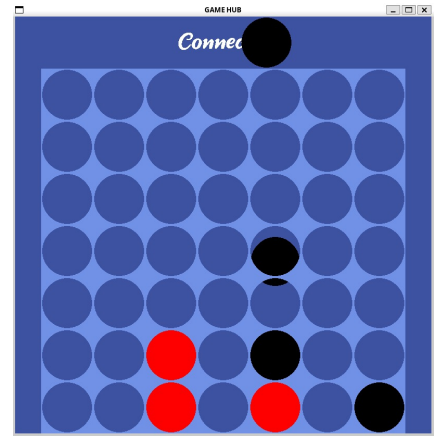


Figure 4: Connect 4

6 Leaderboard and Analytics

After each match, the Python engine appends the result to `history.csv`. The `leaderboard.sh` script then processes this data using `awk` and `sort` to rank players. At the same time, `matplotlib.py` creates visual charts of the players' performance history.

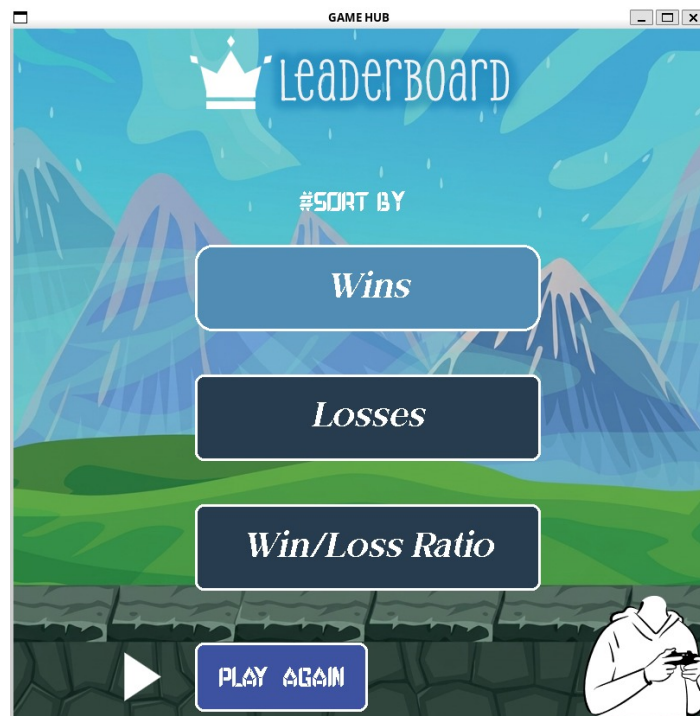


Figure 5: Post-game Leaderboard and Analytics Dashboard

7 Hurdles and Solutions

- Hurdle:** Passing data smoothly between Bash scripts and Python scripts.
Solution: Used `subprocess` and shared CSV files to communicate between languages.
- Hurdle:** Checking win conditions on large grids without slow loops.
Solution: Leveraged NumPy functions like `diagonal()`, `all()`, and `flipplr()` for instant matrix checks.

8 The "10-Hour" Plan

If we had 10 additional hours, we would implement a Single Player mode featuring an machine opponent so users can play even when a second player is not available. definitely we would improve our project .

References

- [1] Python inheritance: <https://docs.python.org/3/tutorial/classes.html>
- [2] Pygame CE Documentation: <https://inventwithpython.com/pygame/>